Phase 4 & Phase 5 - Student Guide

Programming and AI

Welcome

You have already completed the first three phases of the program.

You have learned:

```
Python
   ↓
Programming fundamentals
   ↓
Terminal + Streamlit
   ↓
Building small applications
   ↓
AI / ML fundamentals
```

Now the goal changes.

You are no longer learning programming just to understand programming.

You are going to use what you know to build **web applications** and eventually put one of your own ideas in front of another person.

The goal is not to turn you into a software engineer.

The goal is to make you capable enough to say:

> ***"I have an idea, and I can use code and AI tools to start building it."***

By the end of Phase 5, you should be able to take a simple idea and work toward a **Minimum Working Product (MWP)**.

The Big Picture

Your journey now looks like this:

```
Python
    ↓
"I understand programming."

Terminal / Streamlit
    ↓
"I can build things."

AI / ML
    ↓
"I understand what AI can do."

Django
    ↓
"I can build websites."

HTML / CSS
    ↓
"I can control what the website looks like."

Forms / Application Logic
    ↓
"I can make the website do things."

Small JavaScript Project
    ↓
"I understand what JavaScript is."

Database / SQL
    ↓
"I can make my application remember things."

Authentication
    ↓
"I can work with users."

Deployment
    ↓
"I can put my application on the internet."

Minimum Working Product
    ↓
"I can put my idea in front of a real person."
```

How to Use This Guide

Do not try to memorize everything.

You are learning a different way of working.

For every new concept:

```
Understand the idea
        ↓
Run an example
        ↓
Change something
        ↓
Break something
        ↓
Fix it
        ↓
Build something yourself
```

You are also expected to use AI tools such as ChatGPT.

The goal is **not** to avoid AI.

The goal is to learn how to use AI without becoming dependent on blindly copying code.

PHASE 4

Build Web Applications

Phase 4 Goal

At the end of Phase 4, you should be able to say:

> ***"I understand the basic pieces of a web application, and I can use Django and AI assistance to build a simple website."***

You will learn:

```
Django
HTML
CSS
Templates
URLs
Views
Forms
Models
Database basics
Application logic
Django Admin
Small JavaScript project
```

You do **not** need to become a Django expert.

You need to understand enough that when you see a Django project, it does not look like magic.

---

1. Moving From Cloud Labs to Your Own Computer

Earlier in the program, Cloud Labs and Google Colab helped you avoid spending time installing software.

Starting in Phase 4, you will work on your own computer.

This is intentional.

You already know:

- how to use a terminal
- how Python works
- how packages are installed
- how programs run
- how to work with files

Now you need to learn how a real local development environment works.

---

2. What Is a Virtual Environment?

Different Python projects can require different packages and package versions.

For example:

```
Project A
Django version A
```

while another project might require:

```
Project B
Django version B
```

A virtual environment keeps the dependencies for a project separate.

Think of your computer like this:

```
Your computer
|
├── Project A
|   └── .venv
|
├── Project B
|   └── .venv
|
└── Project C
    └── .venv
```

Each project can have its own environment.

In this program we use:

```
virtualenv
```

rather than Python's built-in `venv` command.

---

3. Creating a Virtual Environment

Open your terminal and enter your project folder.

Then:

```
virtualenv .venv
```

Activate it on Linux/macOS:

```
source .venv/bin/activate
```

You should see something similar to:

```
(.venv) your-name@computer:~/project$
```

The `(.venv)` indicates that the environment is active.

---

4. Installing Project Dependencies

If the project contains:

```
requirements.txt
```

install its dependencies with:

```
python -m pip install -r requirements.txt
```

Using:

```
python -m pip
```

is useful because it makes it clear that pip belongs to the Python environment you are currently using.

---

5. Checking Your Environment

If you are unsure which Python is being used:

```
which python
```

You should see a path similar to:

```
.../your-project/.venv/bin/python
```

You can also check pip:

```
python -m pip --version
```

If these point inside `.venv`, you are using the project's virtual environment.

---

6. Never Commit `.venv`

Your virtual environment belongs to your computer.

It should not be uploaded to Git.

Your `.gitignore` should contain:

```
.venv/
venv/
env/
ENV/
```

If you download or clone a project containing a broken `.venv`, delete it and create a new one.

For example:

```
rm -rf .venv
virtualenv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

A virtual environment can contain paths connected to the location where it was created, so copying one between machines or projects can cause problems.

---

7. Starting a Django Project

A Django project can be started with:

```
django-admin startproject myproject .
```

The exact command depends on the project instructions.

A Django project normally contains files such as:

```
manage.py
myproject/
    settings.py
    urls.py
    ...
```

Do not worry if this initially looks confusing.

You will learn what each important part does.

---

8. Starting an Existing Django Project

When you receive an already-created project, you usually do not create another project.

Instead:

```
cd project-folder
```

activate the environment:

```
source .venv/bin/activate
```

install dependencies:

```
python -m pip install -r requirements.txt
```

run migrations if required:

```
python manage.py migrate
```

then start the server:

```
python manage.py runserver
```

Django normally gives you a local address such as:

```
http://127.0.0.1:8000/
```

Open it in your browser.

---

9. The Django Mental Model

The most important thing to understand is not the individual files.

Understand what happens when a user visits a website.

A simplified Django request looks like:

```
Browser
   ↓
URL
   ↓
Django URL configuration
   ↓
View
   ↓
Application logic
   ↓
Database (if needed)
   ↓
Template
   ↓
HTML response
   ↓
Browser
```

For example:

```
User visits:

/products/

        ↓

Django finds the URL

        ↓

Product view runs

        ↓

View gets products from database

        ↓

Template receives products

        ↓

HTML is generated

        ↓

Browser displays products
```

If you understand this flow, Django becomes much easier to understand.

---

10. manage.py

You will frequently run commands through:

```
python manage.py
```

For example:

```
python manage.py runserver
```

```
python manage.py migrate
```

```
python manage.py makemigrations
```

Think of `manage.py` as a command-line control panel for your Django project.

You do not need to memorize every command.

You can always ask ChatGPT:

> *"What Django management command should I use to do X? Explain what it does before giving me the command."*

---

11. URLs

Django needs to know:

> *"When someone visits this address, what should happen?"*

For example:

```
/
```

might show the homepage.

```
/products/
```

might show products.

```
/contact/
```

might show a contact page.

The URL configuration connects URLs to views.

Conceptually:

```
/products/
     ↓
products view
```

---

12. Views

A view contains the logic that handles a request.

A very simple view might look like:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello!")
```

The important part is not memorizing this code.

Understand:

```
request
   ↓
view
   ↓
response
```

The view receives a request and returns a response.

---

13. Templates

You usually do not want to write an entire HTML page inside Python.

Instead Django can render a template.

Conceptually:

```
Python view
    ↓
data
    ↓
HTML template
    ↓
browser
```

A template might contain:

```
<h1>{{ name }}</h1>
```

If the view provides:

```
name = "Ashish"
```

the browser can receive:

```
<h1>Ashish</h1>
```

This is one of the key ideas in Django.

---

14. Template Variables

Django templates commonly use:

```
{{ variable }}
```

For example:

```
<h1>{{ product.name }}</h1>
<p>{{ product.price }}</p>
```

The view provides the data.

The template decides how the data is displayed.

Think:

```
Python
 ↓
data

Template
 ↓
presentation
```

---

15. Template Conditions

Templates can also contain simple logic.

For example:

```
{% if products %}
    <p>Products are available.</p>
{% else %}
    <p>No products yet.</p>
{% endif %}
```

The template is not full Python.

It provides a simple way to generate HTML based on data.

---

16. Template Loops

For a list:

```
{% for product in products %}
    <h2>{{ product.name }}</h2>
    <p>{{ product.price }}</p>
{% endfor %}
```

The flow is:

```
Python
 ↓
products list
 ↓
template
 ↓
loop
 ↓
HTML for each product
```

---

17. HTML

HTML describes the structure of a web page.

You should become comfortable recognizing:

```
<h1>Heading</h1>

<p>Paragraph</p>

<a href="/">Link</a>

<img src="..." />

<div>Content</div>

<section>Section</section>

<form>...</form>

<input />

<button>Submit</button>
```

You do not need to memorize all HTML.

You need enough familiarity to read AI-generated HTML and make small changes.

---

18. CSS

CSS controls appearance.

Think:

```
HTML
 ↓
What exists?

CSS
 ↓
How does it look?
```

You should recognize properties such as:

```
color
background
margin
padding
width
height
font-size
display
border
```

For example:

```
button {
    padding: 10px 20px;
    font-size: 16px;
}
```

You do not need to become a CSS specialist.

---

19. Why HTML and CSS Matter

You will often ask ChatGPT:

> *"Make my homepage look more professional."*

ChatGPT may return a large amount of HTML and CSS.

You should not need to understand every line.

But you should be able to recognize:

```
This is HTML.
This is CSS.
This changes the layout.
This changes spacing.
This changes the button.
```

That is enough to begin working effectively with AI.

---

20. Forms

Forms allow users to send information to your application.

For example:

```
Name:     [_____________]

Email:    [_____________]

Message:  [_____________]

          [ Submit ]
```

The basic flow is:

```
User enters information
        ↓
Form submitted
        ↓
Django receives data
        ↓
Django validates data
        ↓
Application does something
        ↓
Response
```

---

21. GET and POST

You will encounter:

```
GET
```

and:

```
POST
```

At this level, remember:

```
GET
→ usually asking the server for something

POST
→ usually sending information to the server
```

For example:

```
GET /products/
```

means:

> *"Give me the products page."*

A form submission may use:

```
POST /contact/
```

meaning:

> *"Here is information submitted by the user."*

---

22. CSRF

Django forms often contain:

```
{% csrf_token %}
```

This is a security feature.

You do not need to become a security expert to use it.

Remember:

> *Do not remove CSRF protection just because a form stops working without it.*

If you see a CSRF error, investigate the form and Django configuration rather than disabling security.

---

23. Models

Eventually your application needs to remember information.

For example, a product might have:

```
name
description
price
quantity
```

A Django model describes this data.

For example:

```
class Product(models.Model):
    name = models.CharField(max_length=120)
    price = models.DecimalField(max_digits=10, decimal_places=2)
```

You can think of the model as a description of the information your application stores.

---

24. Model → Database

A simplified relationship is:

```
Django Model
     ↓
Migration
     ↓
Database Table
```

If you change a model, Django needs to know that the database structure must change.

Usually:

```
python manage.py makemigrations
```

creates migration instructions.

Then:

```
python manage.py migrate
```

applies them to the database.

---

25. Migration Mental Model

Imagine you have a blueprint:

```
Product
- name
- price
```

Then you decide:

```
Product
- name
- price
- quantity
```

The database must also change.

A migration records the change.

Think:

```
Model changed
    ↓
makemigrations
    ↓
migration created
    ↓
migrate
    ↓
database updated
```

---

26. Application Logic

A website becomes an application when it starts doing things.

For example:

```
Customer submits booking
        ↓
Django receives booking
        ↓
Validate information
        ↓
Save booking
        ↓
Show confirmation
```

This is application logic.

Do not worry about making it architecturally perfect.

Focus on understanding the flow.

---

27. Django Admin

Django provides an administrative interface.

For example:

```
admin.site.register(Booking)
```

can make a model available through the admin interface.

This is extremely useful for a beginner entrepreneur.

Imagine your website collects:

```
Customer
Email
Booking date
Service
```

You need some way to see those records.

Django Admin can provide that internal interface without you building a separate dashboard first.

---

28. Creating an Admin User

A project may require:

```
python manage.py createsuperuser
```

Django will ask for credentials.

Then you can visit:

```
/admin/
```

and sign in.

The exact URL can depend on the project.

---

29. JavaScript

You are going to learn only a small amount of JavaScript.

This is intentional.

You are **not** taking a JavaScript course.

The goal is:

> *"I understand what JavaScript is, so I won't be scared when I encounter it."*

You already know programming concepts from Python.

JavaScript has similar ideas:

```
variables
functions
conditions
loops
events
```

The syntax is different.

---

30. Python vs JavaScript

Python:

```
name = "Alice"

def greet():
    print("Hello")
```

JavaScript:

```
const name = "Alice";

function greet() {
    console.log("Hello");
}
```

The exact syntax differs.

But the programming concepts are familiar.

That is the point.

---

31. The Browser and the DOM

JavaScript can interact with the page currently displayed in the browser.

The browser represents the HTML page as a structure called the DOM.

A simplified picture:

```
HTML
 ↓
Browser
 ↓
DOM
 ↓
JavaScript
```

JavaScript can respond to events such as:

```
click
input
submit
```

and change what the user sees.

---

32. Small JavaScript Project

A simple project might be:

```
Count: 0

[ - ] [ + ]
```

The flow:

```
User clicks +
        ↓
JavaScript receives event
        ↓
Function runs
        ↓
Count changes
        ↓
DOM updates
        ↓
Browser displays new count
```

This is enough to make the concepts familiar.

---

33. Phase 4 Practice

After each project, make at least one change that was not part of the original instructions.

Examples:

```
Change the heading
Change the CSS
Add a field
Change a button
Add another page
Add another model field
Change the success message
Add a product
Add a validation rule
```

The objective is to move from:

```
"I followed instructions."
```

to:

```
"I can modify an application."
```

---

34. Phase 4 Checkpoint

Before moving to Phase 5, you should be able to explain:

```
☐ What Django is
☐ What a URL does
☐ What a view does
☐ What a template does
☐ What HTML does
☐ What CSS does
☐ What a form does
☐ What a model represents
☐ Why migrations exist
☐ What a database stores
☐ What Django Admin is
☐ What JavaScript does in a browser
```

You do not need to memorize every Django command.

You need to understand the relationships.

PHASE 5

Make Applications Real

Phase 5 takes the website you built in Phase 4 and moves it closer to something a real person could use.

The progression is:

```
Database
 ↓
SQL
 ↓
Authentication
 ↓
Production configuration
 ↓
Deployment
 ↓
Minimum Working Product
```

The goal is:

> ***"I can make an application remember information, work with users, and put it online."***

---

35. Database Thinking

Start with one question:

> ***What does your application need to remember?***

A booking application might remember:

```
Customer
Email
Booking date
Service
Status
```

A product application might remember:

```
Product
Price
Description
Quantity
```

A task application might remember:

```
Task
Owner
Status
Due date
```

---

36. Tables, Rows and Columns

A database table might look like:

```
Products

id | name     | price
---|----------|------
1  | Laptop   | 50000
2  | Keyboard | 2000
3  | Mouse    | 1000
```

Understand:

```
Table
= collection of related records

Row
= one record

Column
= one property of a record

ID
= identifier for a record
```

---

37. Why IDs Matter

Imagine:

```
Laptop
```

and another product also called:

```
Laptop
```

Names do not necessarily uniquely identify records.

An ID gives each record a unique identifier:

```
1
2
3
4
```

You will see IDs throughout web applications and databases.

---

38. SQLite

Many beginner Django projects use SQLite.

The database may appear as:

```
db.sqlite3
```

Django can communicate with SQLite through Python's SQLite support.

You can also use the separate SQLite command-line program to directly query the database.

These are related but different:

```
Python sqlite3 module
        ↓
Python programs communicate with SQLite


sqlite3 command-line program
        ↓
You type SQL directly
```

---

39. Opening SQLite

From the project directory:

```
sqlite3 db.sqlite3
```

You may need to install the SQLite command-line program on Ubuntu:

```
sudo apt install sqlite3
```

Once inside, you will see:

```
sqlite>
```

You are now interacting directly with the database.

---

40. List Tables

Inside SQLite:

```
.tables
```

You may see:

```
django_migrations
django_session
inventory_product
```

The exact table names depend on your project.

---

41. See Table Structure

For example:

```
.schema inventory_product
```

This shows how the table is defined.

This is a useful way to connect:

```
Django model
      ↓
Migration
      ↓
Database table
```

---

42. Display Column Names

SQLite initially displays results without column headings.

Turn headings on:

```
.headers on
```

Use readable columns:

```
.mode column
```

Then:

```
SELECT * FROM inventory_product;
```

The output becomes easier to understand.

---

43. SQL

SQL is the language used to communicate with relational databases.

Do not think:

> *"I need to memorize SQL."*

Think:

> *"I am asking the database a question."*

---

44. SELECT

To get everything:

```
SELECT *
FROM inventory_product;
```

To select specific columns:

```
SELECT name, price
FROM inventory_product;
```

The `*` means:

> *all columns.*

---

45. WHERE

Suppose you want products costing more than ₹5,000:

```
SELECT name, price
FROM inventory_product
WHERE price > 5000;
```

The database returns only matching records.

---

46. ORDER BY

To sort products by price:

```
SELECT name, price
FROM inventory_product
ORDER BY price DESC;
```

`DESC` means descending.

For ascending:

```
SELECT name, price
FROM inventory_product
ORDER BY price ASC;
```

---

47. AND / OR

For multiple conditions:

```
SELECT *
FROM inventory_product
WHERE price > 5000
AND quantity < 20;
```

Or:

```
SELECT *
FROM inventory_product
WHERE price > 5000
OR quantity < 5;
```

---

48. INSERT

You can add a record:

```
INSERT INTO inventory_product
(name, description, price, quantity, created_at)
VALUES
(
    'Headphones',
    'Wireless headphones',
    3000,
    12,
    '2026-08-10 10:00:00'
);
```

The exact fields depend on the project.

---

49. UPDATE

For example:

```
UPDATE inventory_product
SET price = 45000
WHERE id = 1;
```

This is powerful.

Be careful.

Always understand your `WHERE` condition.

Without a `WHERE`, you may update every row.

---

50. DELETE

For example:

```
DELETE FROM inventory_product
WHERE id = 5;
```

Again, understand the `WHERE` condition before running the command.

---

51. SQL Safety Habit

Before:

```
UPDATE ...
```

or:

```
DELETE ...
```

first run the corresponding `SELECT`.

For example, before:

```
DELETE FROM inventory_product
WHERE price < 2000;
```

run:

```
SELECT *
FROM inventory_product
WHERE price < 2000;
```

Confirm that the records are the ones you actually want to modify.

This is a very useful real-world habit.

---

52. Django ORM vs SQL

You may have seen Django code such as:

```
Product.objects.all()
```

and now SQL:

```
SELECT * FROM inventory_product;
```

These are two different ways of working with the same underlying data.

A simplified relationship is:

```
Django ORM
      ↓
Database query
      ↓
Database
```

The ORM lets you work with database data through Python.

SQL lets you communicate directly with the database.

You do not need to become a database engineer.

You need enough understanding to know what is happening.

---

53. Authentication

Once your application has data, you may need to know:

> ***Who is using the application?***

Authentication answers:

> ***Who are you?***

Examples:

```
Login
Logout
Password
Session
User account
```

---

54. Authorization

Authorization answers:

> ***What are you allowed to do?***

For example:

```
User A
 ↓
Can see A's bookings

User B
 ↓
Can see B's bookings
```

A user should not automatically be able to access another user's private data.

Remember:

```
Authentication
= Who are you?

Authorization
= What can you do?
```

---

55. Ownership

When working with user data, ask:

```
Who owns this record?
Who can view it?
Who can edit it?
Who can delete it?
```

This is a simple but powerful security mindset.

For example:

```
Booking
 ├── customer
 ├── date
 └── service
```

If the booking belongs to a user, the application should enforce that ownership where appropriate.

---

56. Hiding Buttons Is Not Security

Suppose you hide:

```
[Delete]
```

from another user.

That does not automatically make the application secure.

A user may still try to directly visit a URL.

Security needs to be enforced on the server.

Think:

```
Browser interface
        ↓
Server validation
        ↓
Database access
```

The server must decide whether the operation is allowed.

---

57. Production Preparation

An application that works on your laptop is not automatically ready for the internet.

Ask:

> *"What changes when another person uses this application?"*

You need to think about:

```
Secrets
Configuration
Debug mode
Allowed hosts
Dependencies
Database
Static files
Deployment
Logs
```

---

58. DEBUG

During development you may use:

```
DEBUG = True
```

Production should generally not expose development debugging information.

The important lesson:

> *Development configuration and production configuration are different.*

Do not copy your local development configuration blindly into production.

---

59. Secrets

Never casually put real secrets into Git.

Examples:

```
SECRET_KEY
API keys
database credentials
service credentials
```

A secret should be supplied through environment/configuration rather than committed as source code.

A common local pattern is:

```
.env
```

while a project may provide:

```
.env.example
```

to document what variables are required without containing real secrets.

---

60. Requirements

Your application depends on packages.

A `requirements.txt` file records the Python dependencies needed by the project.

For example:

```
python -m pip install -r requirements.txt
```

This helps another machine recreate the project's Python environment.

Think:

```
Your code
+
requirements.txt
+
configuration
=
application environment
```

---

61. Deployment

Deployment means making your application available outside your own computer.

The mental model:

```
Your laptop
      ↓
Hosted server
      ↓
Internet
      ↓
User's browser
```

You do not need to understand every detail of cloud infrastructure.

You need to understand the major pieces.

---

62. What a Hosting Environment Needs

A hosted application needs things such as:

```
Your code
Dependencies
Environment variables
Database
Startup command
Static files
Network configuration
```

The exact setup depends on the hosting platform.

Do not worry if the platform's interface changes.

The underlying ideas remain similar.

---

63. Deployment Errors

When deployment fails, do not panic.

Ask:

```
Did the build succeed?

Are dependencies installed?

Are environment variables present?

Can Django start?

Is the database available?

Are migrations applied?

Are static files configured?

What do the logs say?
```

The logs are often your best source of information.

---

64. Read Errors Instead of Fearing Them

A traceback can look intimidating.

Do not read every line immediately.

Look for:

```
Exception type
Error message
File
Line number
```

Then ask:

> *"What was the application trying to do at this point?"*

You can use ChatGPT to explain the traceback.

A good prompt is:

```
I am learning Django.

Here is the error:

[paste error]

Expected:
[what I wanted]

Actual:
[what happened]

Explain the likely cause in beginner-friendly language.

Do not give me a large rewrite.
Tell me what I should check first.
```

---

65. Minimum Working Product

This is the final goal of the program.

A Minimum Working Product is:

> ***The simplest working version of an idea that allows another person to experience its core value.***

It is not:

```
A perfect product
A finished startup
Enterprise software
A production-scale system
Every feature you can imagine
```

It is a useful first version.

---

66. Start With the User

Before writing code, answer:

```
Who is my user?

What problem do they have?

What does my product help them do?

What is the most important action?

What useful result do they get?
```

For example:

```
User:
Local fitness trainer

Problem:
Potential clients cannot easily request a trial session.

Core action:
Request a trial session.

Useful result:
Trainer receives the request.
```

That may be enough for an MWP.

---

67. Define Your Core Journey

Write:

```
Visitor
 ↓
Homepage
 ↓
Understand offer
 ↓
Click CTA
 ↓
Fill form
 ↓
Submit
 ↓
Application saves information
 ↓
Founder sees request
```

This is much more useful than saying:

> *"I want to build a fitness platform."*

---

68. Separate MUST HAVE From LATER

Write two lists.

MUST HAVE

The minimum functionality needed to test the idea.

LATER

Everything that would be useful eventually but is not necessary for the first test.

For example:

MUST HAVE

```
Homepage
Booking form
Save booking
Admin can see booking
Confirmation
```

LATER

```
Payments
Mobile app
Notifications
Analytics
AI recommendations
Multiple languages
Advanced dashboard
```

The goal is to keep the MWP small.

---

69. Do Not Force AI Into Your Product

You have already studied AI/ML.

That does not mean every application needs AI.

Ask:

> *"Does AI create meaningful value for my user?"*

If yes, consider it.

If no, do not add AI just because this is a Programming and AI program.

A simple booking application can be an excellent MWP.

A simple inventory system can be an excellent MWP.

A simple business website can be an excellent MWP.

70. Using ChatGPT as Your Pair Programmer

You are encouraged to use ChatGPT.

The important difference is:

```
Bad workflow


Ask ChatGPT
 ↓
Copy everything
 ↓
Paste
 ↓
Run
 ↓
Error
 ↓
Paste error
 ↓
Copy fix
```

A better workflow:

```
Understand problem
 ↓
Ask ChatGPT for a plan
 ↓
Understand plan
 ↓
Implement one part
 ↓
Test
 ↓
Debug
 ↓
Continue
```

71. Ask for Plans Before Code

Instead of:

> *"Build me a booking website."*

Try:

```
I am building a beginner Django application.

The idea is:
[describe idea]

The user should be able to:
1. ...
2. ...
3. ...

Give me a simple implementation plan.

Tell me:
- what models I need
- what pages I need
- what URLs I need
- what forms I need
- what the user flow is

Do not write the code yet.
```

This helps you understand the architecture before generating code.

---

72. Ask AI to Explain Code

When ChatGPT gives you code:

```
Explain this code to me as a beginner.

Tell me:
1. What this file does.
2. What each important function does.
3. Where the data comes from.
4. Where the data goes.
5. Which parts I am likely to modify later.
```

You do not need to understand every implementation detail immediately.

You do need to understand the important pieces.

---

73. Ask AI to Modify Existing Code

When you have a working project, provide your existing code.

Ask:

```
I have this Django application.

I want to add:
[feature]

Keep the existing structure.

Tell me which files need to change and why.

Do not rewrite unrelated files.
```

This reduces the chance of accidentally replacing your whole application.

---

74. Debugging With AI

When something breaks, provide context.

Use:

```
I am building a Django application.

Expected:
[what should happen]

Actual:
[what happened]

Error:
[paste exact error]

I recently changed:
[describe change]

Relevant code:
[paste relevant code]

Explain the most likely cause.

Tell me what to check first.

Do not give me a complete rewrite.
```

This is much more useful than:

```
fix this
```

---

75. Do Not Hide Errors From Yourself

If ChatGPT fixes an error, ask:

> *"Why did this error happen?"*

Then ask:

> *"How can I recognize this kind of error in the future?"*

Your goal is not to eliminate errors.

Your goal is to become comfortable investigating them.

---

76. Build in Small Steps

Do not ask AI to create a giant application in one response.

Instead:

```
Step 1
Create homepage.

Step 2
Add model.

Step 3
Add migration.

Step 4
Display data.

Step 5
Add form.

Step 6
Save form.

Step 7
Add admin.

Step 8
Add authentication if necessary.

Step 9
Deploy.
```

Small steps are easier to understand and debug.

77. A Useful Builder's Loop

Use this throughout Phase 4 and 5:

```
PLAN
 ↓
BUILD
 ↓
RUN
 ↓
TEST
 ↓
BREAK
 ↓
DEBUG
 ↓
UNDERSTAND
 ↓
IMPROVE
```

Repeat.

78. Don't Be Afraid to Ask "Why?"

When you see:

```
from django.shortcuts import render
```

you can ask:

> *Why are we importing `render`?*

When you see:

```
Product.objects.all()
```

ask:

> *What does this query do?*

When you see:

```
{% csrf_token %}
```

ask:

> *Why is this here?*

When you see:

```
addEventListener()
```

ask:

> *What event is this listening for?*

Curiosity is more valuable than memorization.

---

79. What You Should Know by the End

You do not need to know every line of Django.

You should be able to look at a project and broadly understand:

```
Where the URLs are
Where the views are
Where the models are
Where the templates are
Where the static files are
Where forms are handled
Where the database is involved
Where users are handled
```

You should be able to follow the flow:

```
Browser
  ↓
URL
  ↓
View
  ↓
Model/database
  ↓
Template
  ↓
Browser
```

---

80. What You Don't Need to Learn Yet

You do not need to master:

```
Advanced JavaScript
React
Next.js
Advanced Django
Django REST Framework
Kubernetes
Complex cloud architecture
Advanced database administration
Advanced networking
Advanced security engineering
Distributed systems
Microservices
```

You may learn these later if your product requires them.

For this program, learn the minimum necessary to build your idea.

---

81. When You Encounter Something You Don't Know

This is normal.

Use this process:

```
1. Read the error.
2. Identify the unfamiliar term.
3. Ask ChatGPT to explain it.
4. Read the relevant documentation/tutorial.
5. Try the smallest experiment.
6. Observe what happens.
7. Continue.
```

You do not need to know everything before you start building.

---

82. Your Phase 4 Checklist

Before moving forward, you should have:

```
☐ Created a virtual environment
☐ Activated it
☐ Installed requirements
☐ Started a Django project
☐ Started an existing Django project
☐ Understood manage.py
☐ Understood URLs
☐ Understood views
☐ Created/edited templates
☐ Used HTML
☐ Used CSS
☐ Created a form
☐ Understood GET and POST
☐ Seen CSRF protection
☐ Created a model
☐ Run migrations
☐ Viewed data
☐ Used Django Admin
☐ Completed the small JavaScript project
```

---

83. Your Phase 5 Checklist

You should have:

- ☐ Identified application data
- ☐ Understood tables
- ☐ Understood rows
- ☐ Understood columns
- ☐ Understood IDs
- ☐ Used SQLite
- ☐ Run SELECT queries
- ☐ Used WHERE
- ☐ Used ORDER BY
- ☐ Used INSERT
- ☐ Used UPDATE
- ☐ Used DELETE
- ☐ Understood authentication
- ☐ Understood authorization
- ☐ Considered data ownership
- ☐ Used environment variables
- ☐ Understood production configuration
- ☐ Prepared a Django application for deployment
- ☐ Deployed an application
- ☐ Read deployment logs
- ☐ Tested your application with another person
- ☐ Defined an MWP

---

84. Final Project - Your Minimum Working Product

Now build something connected to your own interests.

It can be:

```
A business idea
A personal project
A small startup experiment
An internal business tool
A portfolio project
A service website
A booking system
A simple marketplace
A content website
A productivity tool
```

The important thing is that another person should be able to use the core functionality.

---

85. MWP Planning Template

Fill this in before coding.

Idea

```
My idea:
```

Target user

```
My target user:
```

Problem

```
The problem they have:
```

Core action

```
The most important action they should be able to perform:
```

Useful result

```
After performing that action, the user gets:
```

Data

```
What does my application need to remember?
```

Pages

```
1.
2.
3.
4.
```

MUST HAVE

```
1.
2.
3.
4.
```

LATER

```
1.
2.
3.
4.
```

---

86. MWP Development Plan

Use this sequence:

```
1. Write the idea.
2. Define the user.
3. Define the core problem.
4. Define the core user journey.
5. Decide what data must be stored.
6. Build the simplest homepage.
7. Build the core form/action.
8. Save the important data.
9. Add authentication only if needed.
10. Add admin/internal access where useful.
11. Test the complete flow.
12. Deploy.
13. Give the URL to another person.
14. Observe what happens.
15. Record feedback.
16. Decide what to build next.
```

---

87. The Final Test

Your application does not need to be impressive.

It needs to work.

Ask another person to use it without explaining every step.

Watch them.

Do not immediately help.

Notice:

```
Where do they get confused?
Where do they stop?
What do they expect to happen?
What do they click?
What do they ask?
```

This feedback is extremely valuable.

---

88. Your Final Demonstration

When presenting your MWP, explain:

1. Problem

> *What problem are you solving?*

2. User

> *Who is the product for?*

3. Product

> *What did you build?*

4. Core journey

Show:

```
Start
 ↓
Action
 ↓
Result
```

5. Technology

Briefly explain:

```
Django
Database
Authentication, if used
JavaScript, if used
AI, if used
Deployment
```

6. AI usage

Explain:

> *"Where did I use ChatGPT?"*

and:

> *"What did I understand myself versus what did AI help me implement?"*

7. Next step

Explain:

> *"What would I build next if I had another week?"*

---

89. What Success Looks Like

Success is not:

> *"I memorized Django."*

Success is not:

> *"I can write 500 lines of JavaScript."*

Success is not:

> *"I can deploy Kubernetes."*

Success is:

> ***"I have an idea, and I know how to start building it."***

You should leave this program comfortable with:

```
Idea
 ↓
Product thinking
 ↓
Code
 ↓
AI assistance
 ↓
Testing
 ↓
Debugging
 ↓
Database
 ↓
Web application
 ↓
Deployment
 ↓
Real person
```

---

90. Your New Relationship With Programming

Earlier, programming may have felt like:

```
"I need to know everything before I can build."
```

That is not how modern building works.

Instead:

```
"I know enough to understand the problem."
        ↓
"I can research what I don't know."
        ↓
"I can ask AI for help."
        ↓
"I can test the result."
        ↓
"I can debug when it fails."
        ↓
"I can keep going."
```

That is the skill we want you to develop.

---

91. The Builder Mindset

When you encounter a new technology, do not immediately think:

> *"I have to learn the entire technology."*

Ask:

> *"What is the minimum I need to know to use this for my current problem?"*

For example:

```
Need a database?
→ Learn enough SQL/database concepts.

Need login?
→ Learn enough authentication.

Need JavaScript?
→ Learn enough DOM/events.

Need deployment?
→ Learn enough hosting/configuration.

Need AI?
→ Learn enough to understand the model and use it responsibly.
```

This keeps you moving.

---

92. Your Most Important Skill

The most valuable skill from this program may not be Python.

It may not be Django.

It may not be SQL.

It may not even be AI.

It is:

> ***The ability to turn an unfamiliar technical problem into a sequence of smaller questions.***

For example:

Instead of:

> *"How do I build my startup?"*

ask:

```
What is my user's problem?
        ↓
What is the core action?
        ↓
What page do they need?
        ↓
What data do I need?
        ↓
What form do I need?
        ↓
Where should the data be stored?
        ↓
Who can access it?
        ↓
How do I test it?
        ↓
How do I deploy it?
```

Now the problem is manageable.

---

93. Final Principle

You are not being trained to become a software engineer.

You are being trained to become a **technically capable builder**.

You should be able to sit down with:

```
An idea
+
A laptop
+
A terminal
+
ChatGPT
```

and think:

> ***"I can start."***

That is the goal of Phase 4 and Phase 5.